Software Architecture and Design Specification

Project: Student Attendance Management System

Version: 1.0

Authors: Greeshma, Sanjit, Chandraraj Patil, Chethan BJ

Date: 01-10-2026

Status: For Review

1. Introduction

1.1 Purpose
This document specifies the architecture and design of the Student Attendance Management System.

1.2 Scope
Covers authentication, attendance capture and correction, attendance history, reporting, administration, and audit functions.

1.3 Audience
Developers, QA engineers, system administrators, instructors, and assessment evaluators.

1.4 Definitions
SAM, RBAC, API, REST, CSV, HTTPS, TLS, RTM.

2. Document Overview

2.1 How to use this document
Provides architectural deliverables including UML component and sequence diagrams, architecture rationale, security design, API design, and traceability.

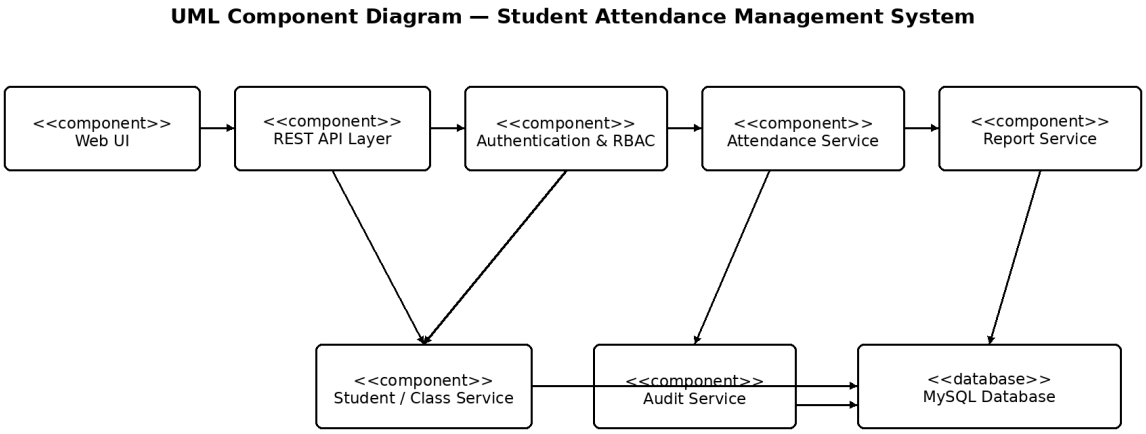
2.2 Related Documents
SRS, STP, RTM.

3. Architecture

3.1 Goals & Constraints
Goals: accurate attendance, secure access, reliable reporting, maintainability, and responsive operation. Constraints: role-based access, relational data consistency, configurable attendance policy, and secure deployment.

3.2 Stakeholders & Concerns
Students: privacy and easy access to attendance.
Faculty: fast and accurate marking and reporting.
Administrators: maintainability and account/class management.
Developers: modularity and testability.

3.3 Component (UML) Diagram



3.4 Component Descriptions

- Web UI: provides student, faculty, and administrator screens.
- Authentication & RBAC: verifies credentials and authorizes roles.
- Attendance Service: records, updates, and retrieves attendance.
- Report Service: calculates percentages, flags low attendance, and exports reports.
- Student & Class Service: manages rosters, courses, classes, and enrollments.
- REST API Layer: connects UI requests to application services.
- Database: stores users, students, classes, enrollments, attendance, and audit data.

3.5 Chosen Architecture Pattern and Rationale

Layered architecture is chosen for separation of presentation, API, business services, and data access. It keeps attendance logic modular and makes testing and maintenance easier. A microservices architecture is not necessary for the expected project scale.

3.6 Technology Stack & Data Stores

Proposed stack: React-based web UI, Node.js/Express REST APIs, MySQL relational database, HTTPS/TLS, and JWT/session-based authentication. The stack can be replaced if the implementation team adopts equivalent technologies.

3.7 Risks & Mitigations

Risk: incorrect or duplicate attendance records → Mitigation: validation, unique constraints, and audit logging.

Risk: unauthorized data access → Mitigation: RBAC and server-side authorization.
Risk: database failure → Mitigation: scheduled backups and recovery testing.

3.8 Traceability to Requirements

SAM-F-001, SAM-F-002 → Authentication & RBAC
SAM-F-003..SAM-F-007 → Attendance Service
SAM-F-008..SAM-F-013 → Attendance History & Report Service
SAM-F-014..SAM-F-017 → Administration & Dashboard Services

3.9 Security Architecture

Threat Modeling (STRIDE): Spoofing → authenticated login and session controls; Tampering → server-side validation and audit trail; Information disclosure → HTTPS/TLS and authorization; Denial of Service → rate limiting and input validation; Elevation of privilege → RBAC and protected API endpoints.

4. Design

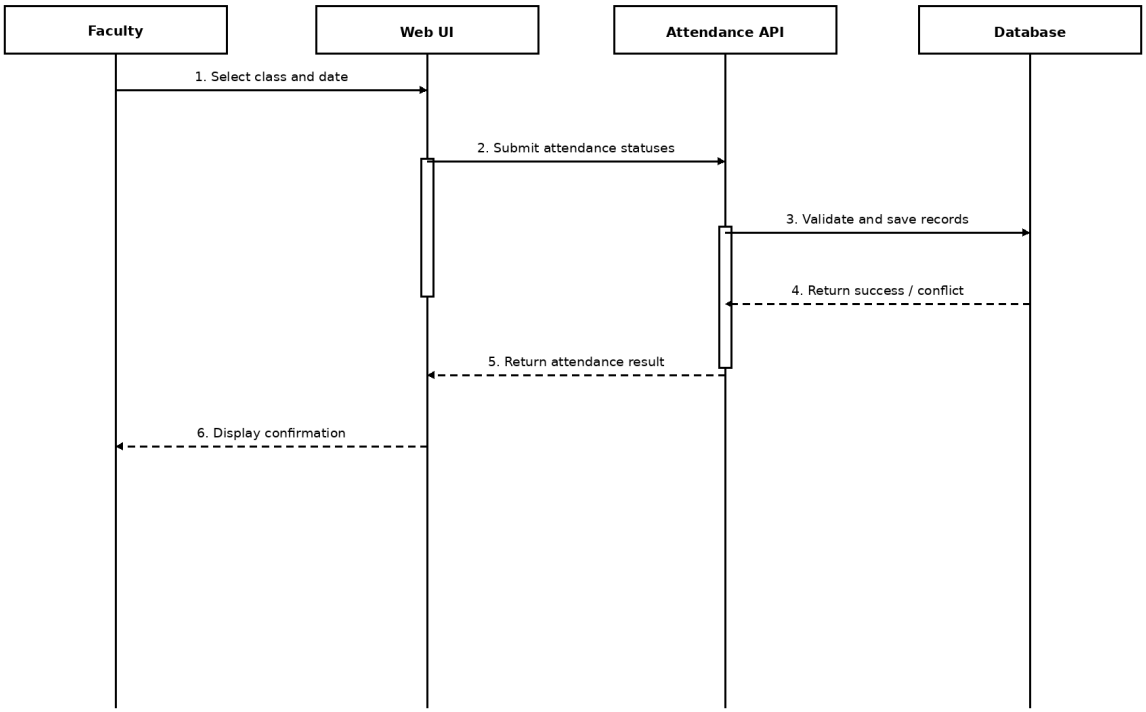
4.1 Design Overview

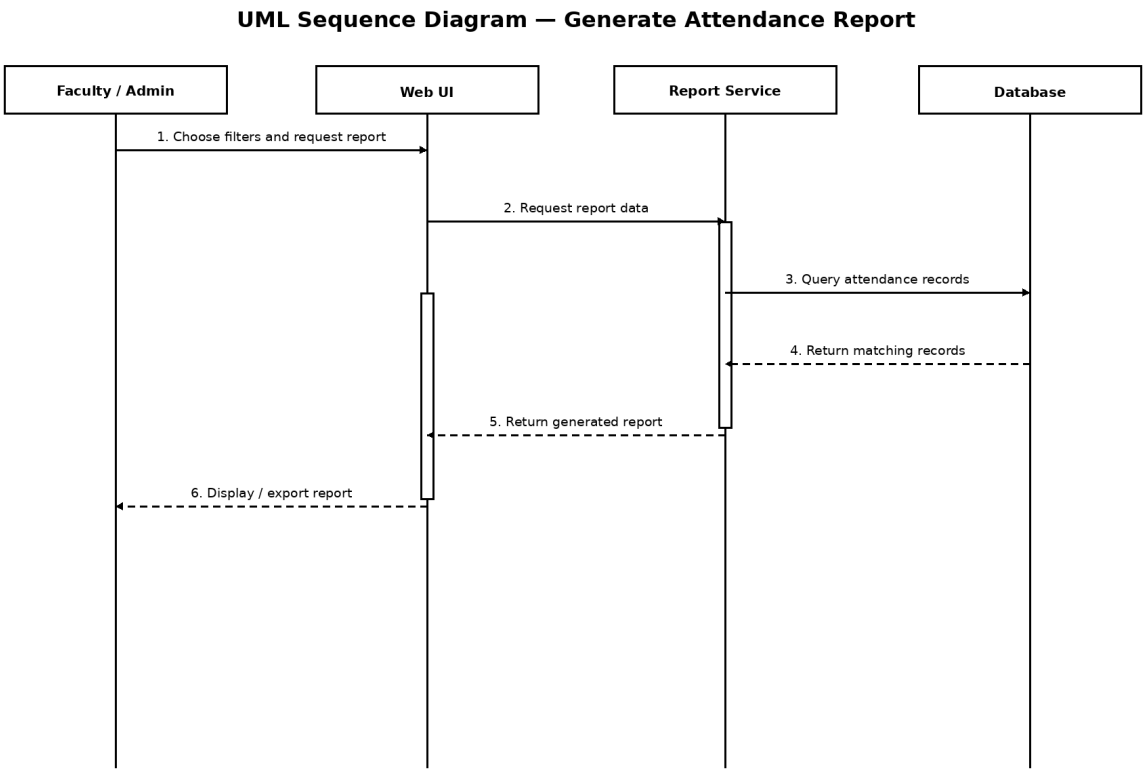
The system uses layered modules for authentication, attendance processing, reporting, administration, and persistence. Business rules remain server-side so that client-side changes cannot bypass authorization or attendance validation.

4.2 UML Sequence Diagrams

Two sequence diagrams cover project-specific flows: marking attendance and generating an attendance report.

UML Sequence Diagram — Mark Attendance





4.3 API Design

Interface definitions for the authentication and attendance/report components.

Endpoint: /api/auth/login
Method: POST
Request: {username, password}
Response: {token, role}
Errors: 401 Invalid credentials, 403 Access denied.

Endpoint: /api/attendance
Method: POST
Request: {classId, date, records[]}
Response: {status, savedCount}
Errors: 400 Invalid data, 409 Duplicate attendance.

Endpoint: /api/reports/attendance
Method: GET
Request: {classId, from, to}
Response: {summary, records, lowAttendance}
Errors: 400 Invalid filter, 403 Access denied.

4.4 Error Handling, Logging & Monitoring

Standardized error responses are returned for validation, authentication, authorization, conflict, and server errors. Passwords are never logged. Audit logs capture attendance corrections and administrative changes. Monitoring includes API error rate, response time, authentication failures, and report generation failures.

4.5 UX Design

The web UI uses clear role-specific dashboards, readable attendance tables, confirmation before bulk changes, visible validation messages, responsive layouts, and accessible controls.

4.6 Open Issues & Next Steps

Future enhancements: biometric attendance integration, timetable-based attendance sessions, scheduled email reports, notifications for low attendance, and integration with institutional ERP systems.

5. Appendices

5.1 Glossary: SAM, RBAC, API, REST, CSV, HTTPS, TLS, RTM.
5.2 References: SRS, STP, UML/IEEE 42010 architectural concepts, OWASP application-security guidance.
5.3 Tools: PlantUML/Graphviz, draw.io, Swagger/OpenAPI.